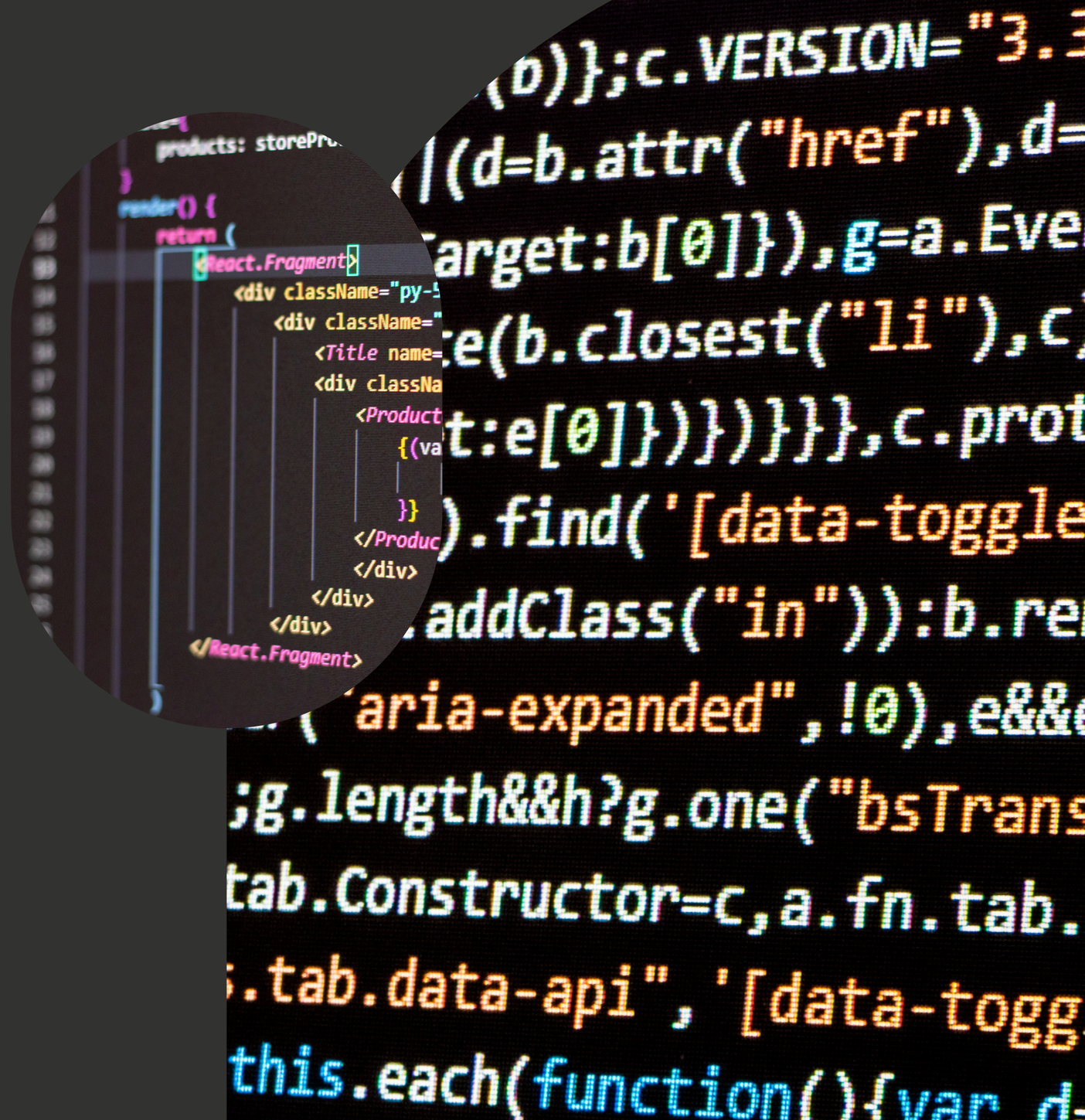
SZYFR MENDELEJEWA

AUTOR: NIKODEM PŁOWUCHA

DATA: 23.11.2025



1. TEMAT PROJEKTU

PROJEKT POLEGA NA IMPLEMENTACJI SZYFRU MENDELEJEWA – ALGORYTMU, KTÓRY ZAMIENIA LITERY TEKSTU NA NUMERY ATOMOWE PIERWIASTKÓW CHEMICZNYCH ODPOWIADAJĄCYCH ICH SYMBOLOM. SYMBOLE PIERWIASTKÓW MOGĄ BYĆ JEDNO- LUB DWULITEROWE.

```c
// Definicja struktury przechowującej symbol i numer atomowy
typedef struct {
    const char* symbol;
    int number;
} Element;

// Słownik 118 pierwiastków chemicznych
Element elements[] = {
    {"h",1},{"he",2},{"li",3},{"be",4},{"b",5},{"c",6},{"n",7},{"o",8},{"f",9},{"ne",10},
    {"na",11},{"mg",12},{"al",13},{"si",14},{"p",15},{"s",16},{"cl",17},{"ar",18},{"k",19},{"ca",20},
    {"sc",21},{"ti",22},{"v",23},{"cr",24},{"mn",25},{"fe",26},{"co",27},{"ni",28},{"cu",29},{"zn",30},
    {"ga",31},{"ge",32},{"as",33},{"se",34},{"br",35},{"kr",36},{"rb",37},{"sr",38},{"y",39},{"zr",40},
    {"nb",41},{"mo",42},{"tc",43},{"ru",44},{"rh",45},{"pd",46},{"ag",47},{"cd",48},{"in",49},{"sn",50},
    {"sb",51},{"te",52},{"i",53},{"xe",54},{"cs",55},{"ba",56},{"la",57},{"ce",58},{"pr",59},{"nd",60},
    {"pm",61},{"sm",62},{"eu",63},{"gd",64},{"tb",65},{"dy",66},{"ho",67},{"er",68},{"tm",69},{"yb",70},
    {"lu",71},{"hf",72},{"ta",73},{"w",74},{"re",75},{"os",76},{"ir",77},{"pt",78},{"au",79},{"hg",80},
    {"tl",81},{"pb",82},{"bi",83},{"po",84},{"at",85},{"rn",86},{"fr",87},{"ra",88},{"ac",89},{"th",90},
    {"pa",91},{"u",92},{"np",93},{"pu",94},{"am",95},{"cm",96},{"bk",97},{"cf",98},{"es",99},{"fm",100},
    {"md",101},{"no",102},{"lr",103},{"rf",104},{"db",105},{"sg",106},{"bh",107},{"hs",108},{"mt",109},
    {"ds",110},{"rg",111},{"cn",112},{"nh",113},{"fl",114},{"mc",115},{"lv",116},{"ts",117},{"og",118}
};

int element_count = sizeof(elements) / sizeof(Element);

// Użyta funkcja: find_element() – szuka symbolu pierwiastka w tablicy
int find_element(const char* sym) {
    for (int i = 0; i < element_count; i++) {
        if (strcmp(elements[i].symbol, sym) == 0)
            return elements[i].number;
    }
    return -1;
}
```

2. OPIS POBIERANYCH DANYCH

PROGRAM POBIERA OD UŻYTKOWNIKA CIĄG ZNAKÓW (TEKST ZŁOŻONY Z LITER A–Z). DANE WEJŚCIOWE MOGĄ ZAWIERAĆ WIELE SŁÓW ROZDZIELONYCH SPACJĄ.

OPIS POBIERANYCH DANYCH

FRAGMENTY Z FUNKCJI MAIN

```
// --- Walidacja Startowa (Menu Wyboru) ---
while (1) {
    printf("\n### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###\n");
    printf("1. Rozpocznij szyfrowanie\n");
    printf("2. Wyjdz z programu\n");
    printf("Wybierz opcje (1 lub 2): ");

    if (scanf("%d", &choice) != 1) {
        while (getchar() != '\n');
        printf("Nieprawidlowy format wejscia. Sprobuj ponownie.\n");
        continue;
    }

    while (getchar() != '\n');

    if (choice == 1) {
        break;
    }
    else if (choice == 2) {
        printf("Program zostal zakonczony.\n");
        exit(0);
    }
    else {
        printf("Nieprawidlowy wybor. Wpisz 1 lub 2.\n");
    }
}
// --- Koniec Walidacji Startowej ---
```

OPIS POBIERANYCH DANYCH

FRAGMENTY Z FUNKCJI MAIN

```c
// Pobieranie tekstu
printf("\n--- Rozpoczeto Szyfrowanie ---\n");
printf("Podaj tekst do zaszyfrowania: ");
// Wczytywanie wejścia
if (fgets(text, sizeof(text), stdin) == NULL) {
    text[0] = '\0'; // Zabezpieczenie na wypadek błędu wczytywania
}
else {
    text[strcspn(text, "\n")] = 0;
}

// Walidacja znaków wejściowych
for (i = 0; text[i] != '\0'; i++) {
    if (!isalpha(text[i]) && !isspace(text[i])) {
        printf("Blad walidacji danych: Tekst moze zawierac tylko litery i spacje. Znaleziono niedozwolony znak: '%c'.\n", text[i]);
        return 1;
    }
}
```

3. OPIS OTRZYMYWANYCH REZULTATÓW

PROGRAM ZWRACA TEKST ZASZYFROWANY, GDZIE KAŻDA LITERA LUB PARA LITER ZOSTAJE ZASTĄPIONA LICZBĄ ATOMOWĄ PIERWIASTKA. GWIAZDKA (*) ODDZIELA LICZBY W RAMACH JEDNEGO SŁOWA, A PODWÓJNA GWIAZDKA (**) ODDZIELA KOLEJNE SŁOWA.

OPIS OTRZYMYWANYCH REZULTATÓW

```
    // Kopiowanie tekstu (strtok modyfikuje oryginalny ciąg)
    strcpy(temp_text, text);
    // Inicjalizacja tokenizacji
    word = strtok(temp_text, " ");

    // Główna pętla szyfrowania słów
    while (word != NULL) {
        // Resetowanie licznika dla nowego słowa
        out_len = 0;
        if (strlen(word) == 0) {
            word = strtok(NULL, " ");
            continue;
        }
        if (!encode_word(word, 0, strlen(word), output, &out_len)) {
            success = 0;
            break;
        }
        if (!first_word) printf("**");
        for (i = 0; i < out_len; i++) {
            printf("%d", output[i]);
            if (i < out_len - 1) printf("*");
        }

        first_word = 0;
        word = strtok(NULL, " ");
    }
    if (!success) {
        printf("\nNie da sie zaszyfrowac podanego tekstu tym sposobem.\n");
    }
    else {
        printf("\n");
    }
    return 0;
}
```

4. OPIS ALGORYTMU

ALGORYTM DZIAŁA REKURENCYJNIE. DLA KAŻDEJ POZYCJI W SŁOWIE:

- PRÓBUJE DOPASOWAĆ SYMBOL DWULITEROWY.
- JEŚLI NIE PASUJE – PRÓBUJE SYMBOL JEDNOLITEROWY.
- JEŚLI DOPASOWANIE JEST MOŻLIWE – DODAJE LICZBĘ ATOMOWĄ DO BUFORA.
- JEŚLI NIE – WRACA (BACKTRACKING).

JEŚLI REKURENCJA DOJDZIE DO KOŃCA SŁOWA – SŁOWO JEST POPRAWNIE SZYFROWALNE.

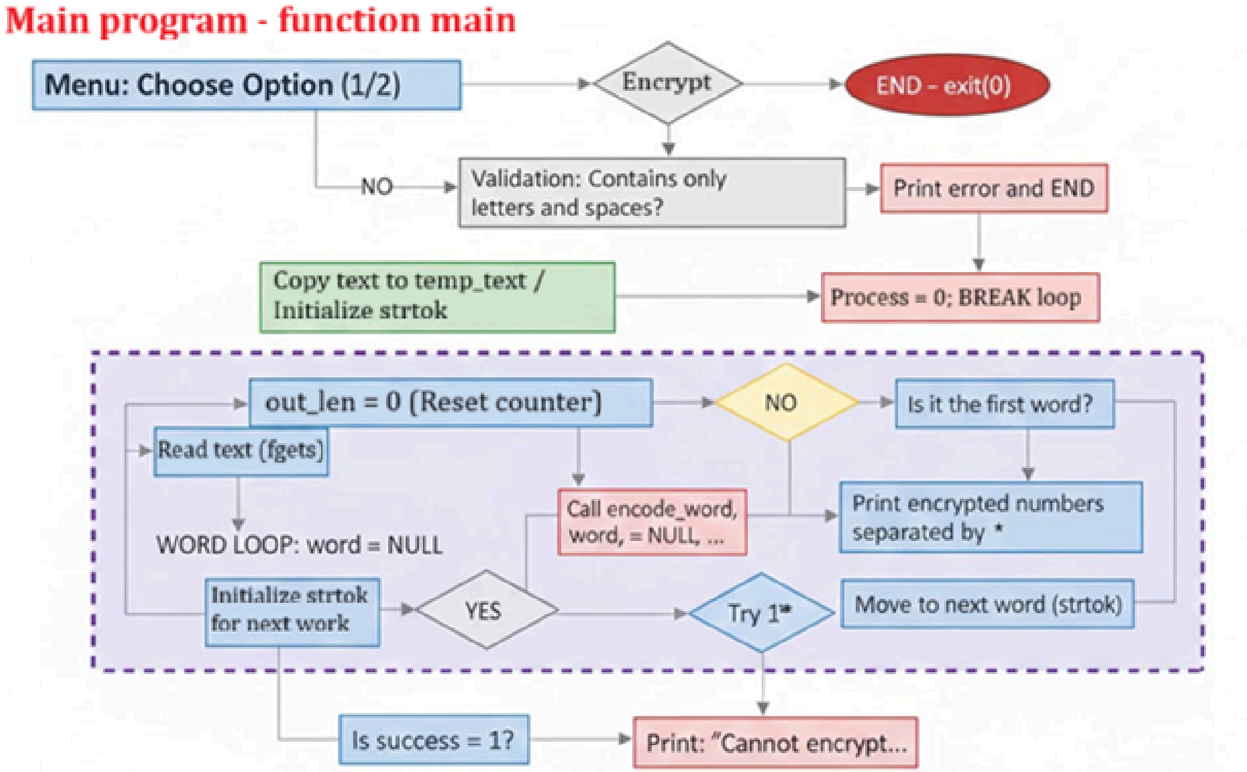
ALGORYTM

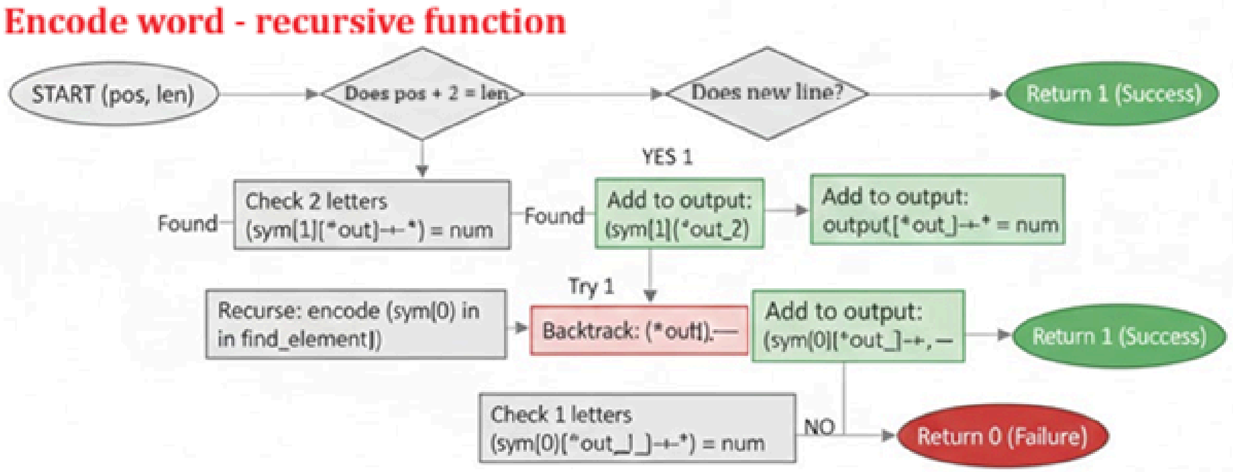
```c
// Rekurencyjna funkcja próbująca zakodować słowo
int encode_word(const char* word, int pos, int len, int output[], int* out_len) {
    if (pos == len) return 1; // sukces
    char sym[3] = { 0 };

    // spróbuj 2-literowy symbol
    if (pos + 2 <= len) {
        sym[0] = tolower(word[pos]);
        sym[1] = tolower(word[pos + 1]);
        int num = find_element(sym);
        if (num != -1) {
            output[(*out_len)++] = num;
            if (encode_word(word, pos + 2, len, output, out_len))
                return 1;
            (*out_len)--; // cofamy (backtracking)
        }
    }

    // spróbuj 1-literowy symbol
    sym[0] = tolower(word[pos]);
    sym[1] = '\0';
    int num = find_element(sym);
    if (num != -1) {
        output[(*out_len)++] = num;
        if (encode_word(word, pos + 1, len, output, out_len))
            return 1;
        (*out_len)--; // cofamy (backtracking)
    }
    return 0;
}
```

SCHEMAT BLOKOWY



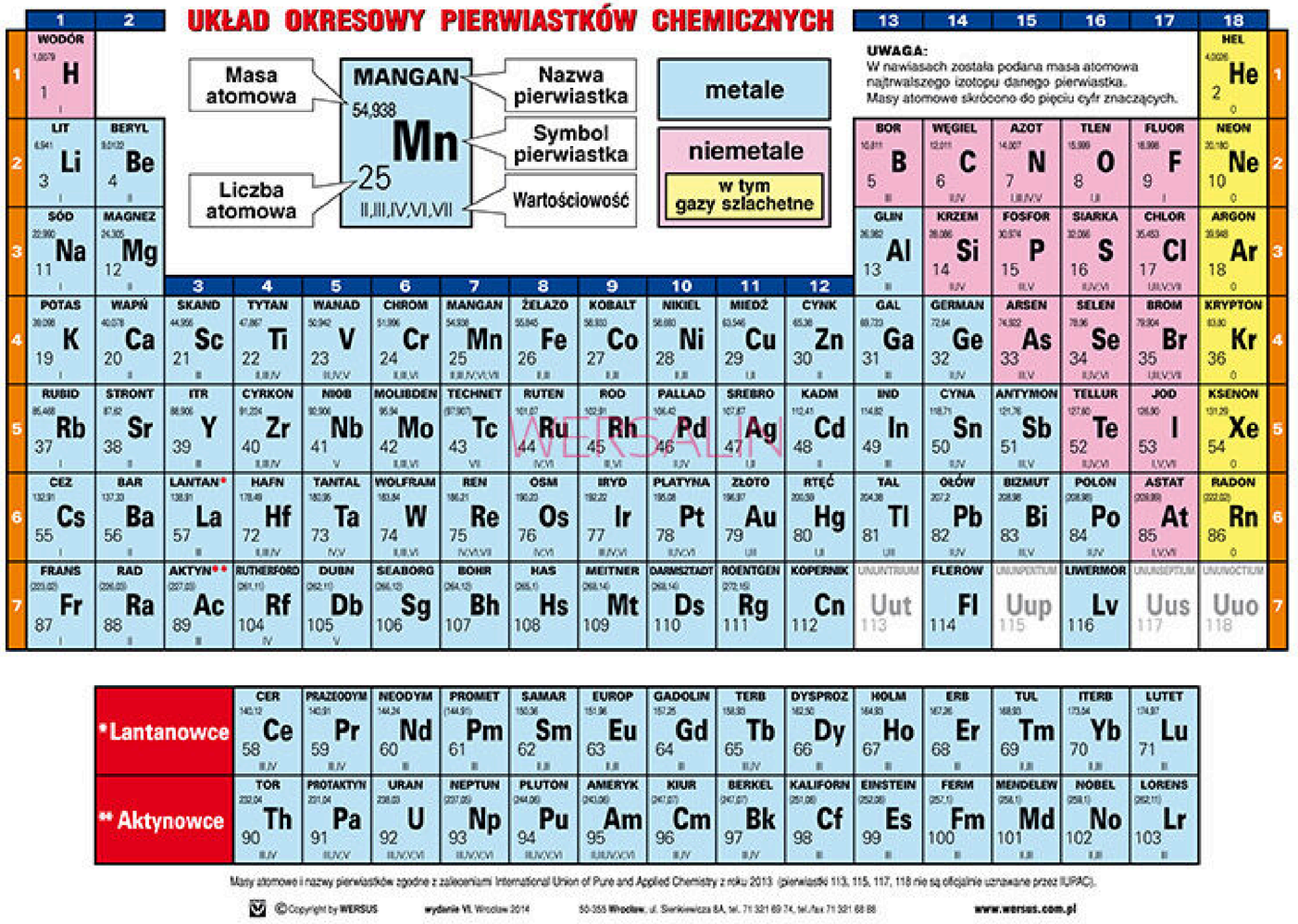


FUNKCJE

UŻYTE FUNKCJE:

- FIND_ELEMENT() – SZUKA SYMBOLU PIERWIASTKA W TABLICY
- ENCODE_WORD() – REKURENCYJNIE SZYFRUJE SŁOWO
- STRTOK() – DZIELI TEKST NA SŁOWA
- TOLOWER() – NORMALIZUJE WIELKOŚĆ LITER

OKRESOWY UKŁAD PIERWIASTKÓW



5. TESTY POPRAWNOŚCI DZIAŁANIA

WEJŚCIE: SOBOTA RANO

WYJŚCIE: 16*8*5*8*7388*102**

```
### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###
1. Rozpocznij szyfrowanie
2. Wyjdz z programu
Wybierz opcje (1 lub 2): 1

--- Rozpoczeto Szyfrowanie ---
Podaj tekst do zaszyfrowania: SobOta rANo
16*8*5*8*73**88*102
```

TESTY POPRAWNOŚCI DZIAŁANIA

WEJŚCIE: HE NA

WYJŚCIE: 211**

```
### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###
1. Rozpocznij szyfrowanie
2. Wyjdz z programu
Wybierz opcje (1 lub 2): 1

--- Rozpoczeto Szyfrowanie ---
Podaj tekst do zaszyfrowania: he na
2**11
```

TESTY POPRAWNOŚCI DZIAŁANIA

PRZYPADKI BŁĘDNE:

WEJŚCIE: KOT

WYJŚCIE: NIE DA SIĘ ZASZYFROWAĆ PODANEGO TEKSTU TYM SPOSOBEM.

```
### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###
1. Rozpocznij szyfrowanie
2. Wyjdz z programu
Wybierz opcje (1 lub 2): 1

--- Rozpoczeto Szyfrowanie ---
Podaj tekst do zaszyfrowania: kot

Nie da sie zaszyfrowac podanego tekstu tym sposobem.
```

6. WNIOSKI

PROJEKT POZWOLIŁ ZROZUMIEĆ DZIAŁANIE REKURENCJI, BACKTRACKINGU ORAZ PRACĘ Z TABLICAMI STRUKTUR.

NAUCZYŁEM SIĘ TEŻ OBSŁUGI WEJŚCIA ZNAKOWEGO I ANALIZOWANIA DANYCH POD KĄTEM WYJĄTKÓW W ALGORYTMIE.

DZIĘKUJE ZA UWAGĘ

I WCALE NIE PRÓBUJĘ ZACHĘCIĆ DO KUPNA PROGRAMU (WCALE)

Nikodem Płowucha - RMS
(1.semestr), Programowanie I

